COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Aug 4, 2026	**DATE SUBMITTED:**	Aug 25, 2026
NAME:	Ellaiza C. Vizcayno	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5 **Queues**

1. PROCEDURE OUTPUT

USING THE QUEUE STL

ILOA > main.cpp > ...

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main(){
    //create an object
    std::queue<char> myQ;

    myQ.push('E');
    myQ.push('L');
    myQ.push('L');
    myQ.push('A');
    myQ.push('I');
    myQ.push('Z');
    myQ.push('A');
    display(myQ);

    //try all member functions:
    //empty
    std::cout<<"is Queue empty? : ";
        if(myQ.empty()){
            std::cout<<"YES"<<std::endl;
        }else{
            std::cout<<"NO"<<std::endl;
        }
    //size
    std::cout<<"Size of Queue: "<<myQ.size()<<std::endl;

    //front
    std::cout<<"Front of Queue: "<<myQ.front()<<std::endl;
```

```cpp
    //back
    std::cout<<"Back of Queue: "<<myQ.back()<<std::endl;

    //pop
    std::cout<<"Popping.. "<<std::endl;
    myQ.pop();

    //display
    std::cout<<"After Popping:"<<std::endl;
    display(myQ);

    //push
    std::cout<<"Pushing X.. "<<std::endl;
    myQ.push('X');

    //display
    std::cout<<"After Pushing X: "<<std::endl;
    display(myQ);

    return 0;
}
void display(std::queue<char> copyQ){
    //creates a copy of the queue that was passed
    std::queue<char> c = copyQ;

    while(!c.empty()){
        std::cout<<" "<<c.front();
        c.pop();
    }
    std::cout<<"\n";
}
```

OUTPUT:	OBSERVATIONS:
E L L A I Z A is Queue empty? : NO Size of Queue: 7 Front of Queue: E Back of Queue: A Popping.. After Popping: L L A I Z A Pushing X.. After Pushing X: L L A I Z A X	The STL library is easy to use but is missing a function to display all elements in the queue. The function names are also self-explanatory. The push function is used to assign an element into the queue, this element is inserted at the back. Although if there is no current element at the front, it becomes the front element. The empty function returns a bool value and is used to check if the queue is empty. The size of the queue returns the number of elements inside the queue. The front and back functions are used to show the front and back elements respectively. Then to display all elements in the queue, we use the front and pop function to traverse the queue.

USING A SELF-MADE TEMPLATE FOR QUEUE

MAIN CODE:

main.cpp

ILOB > main.cpp > main()

```cpp
#include <iostream>
// #include "queueArrLL.h"
#include "queueLL.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('E', &front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;
    enqueue('L', &front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;
    enqueue('A', &front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;

    dequeue(&front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;
    dequeue(&front, &back);
    dequeue(&front, &back);

    enqueue('E', &front, &back);
    enqueue('L', &front, &back);
    enqueue('A', &front, &back);
    start(&front);
    end(&back);
    dsiplay(&front, &back);

    return 0;
}
```

TEMPLATE FILE:

queueLL.h

ILOB > queueLL.h > dsiplay<T>(qNode<T>**, qNode<T>**)

```
1   #ifndef QUEUELL_H
2   #define QUEUELL_H
3
4   //create a class the node
5   template <typename T>
6   class qNode{
7       public:
8           T data;
9           qNode<T>* next;
10  };
11
12  //creates the new node
13  template <typename T>
14  qNode<T>* new_node(T newData){
15      qNode<T>* newNode = new qNode<T>;
16      newNode->data = newData;
17      newNode->next = nullptr;
18      return newNode;
19  }
20  //enqueue operation
21  template <typename T>
22  void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
23      //create new node
24      qNode<T>* newNode = new_node(newData);
25
26      //inserting into an empty queue
27      if((*frontPtr)==nullptr && (*backPtr)==nullptr){
28          (*frontPtr) = newNode;
29          (*backPtr) = newNode;
30      }
31
32      //insert item into non empty queue
33      (*backPtr)->next = newNode;
34      (*backPtr) = newNode;
35  }
```

```cpp
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    //add a check if quque is already empty
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        std::cout<<"Queue is empty.\n";
        return;
    }

    //create a temp node
    qNode<T>* deleteNode = nullptr; // temp var
    deleteNode = (*frontPtr);

    //check if queue has 1 node
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }

    //delete node
    (*frontPtr) = (*frontPtr)->next;
    deleteNode->next = nullptr;
    delete deleteNode;
}
//check the front
template <typename T>
void start(qNode<T>** frontPtr){
    qNode<T>* temp = (*frontPtr);
    if((*frontPtr) == nullptr){
        std::cout<<"Queue is empty."<<std::endl;
        return;
    }
    std::cout<<"Front of Queue: "<<temp->data<<std::endl;

}
```

```
73  //check the back
74  template <typename T>
75  void end(qNode<T>** backPtr){
76      if((*backPtr) == nullptr){
77          std::cout<<"Queue is empty."<<std::endl;
78          return;
79      }
80      qNode<T>* temp = (*backPtr);
81      std::cout<<"Back of Queue: "<<temp->data<<std::endl;
82  }
83  //display all
84  template <typename T>
85  void dsiplay(qNode<T>** frontPtr, qNode<T>** backPtr){
86      if((*frontPtr) == nullptr && (*backPtr) == nullptr){
87          std::cout<<"Queue is empty.\n";
88          return;
89      }
90      qNode<T>* temp = (*frontPtr);
91      while(temp != nullptr){
92          std::cout<<temp->data<<" ";
93          temp = temp->next;
94      }
95  }
96
97  #endif
```

OUTPUT:	OBSERVATIONS:
E E E L E A L A A A Queue is empty. Front of Queue: E Back of Queue: A E L A	Creating your own template allows you to control the output you want, so when it comes to understanding what each data structure does, it's really helpful. I just had a bit of a problem when it came to the parameters needed for each function. Also, when it came to naming functions, I had forgotten that I had already used the front and back so I had to change the function names when it comes to displaying the front and back. During the activity, I also came across problems with pointers and temporary variables. It just means I needed to familiarize myself more with pointers to really get used to understanding it. Another thing to note is to increase my tolerance to noisy seatmates especially to the one to my right.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Source Code

```cpp
#include <iostream>
#include "queueArr.h"

class Job{
    public:
        int jobID;
        std::string userName;
        int numberPages;

        Job(){
            jobID = 0;
            userName = "";
            numberPages = 0;
        }

        Job(int id, std::string name, int pages){
            jobID = id;
            userName = name;
            numberPages = pages;
        }
};

class Printer{
    private:
        queueArr<Job> jobQueue;

    public:
        Printer(size_t capacity) : jobQueue(capacity) {}

        void addJob(Job newJob){
            jobQueue.Enqueue(newJob);
            std::cout << "Added Job " << newJob.jobID << " (" << newJob.userName
                      << ", " << newJob.numberPages << " pages)\n";
        }
        void clearJobs(){
            jobQueue.Clear();
        }
        void showJob(){
            if(jobQueue.Empty()){
                std::cout << "Now Processing: (no jobs in queue)\n";
            }
            else{
                Job current = jobQueue.Front();
                std::cout << "Now Processing: Job " << current.jobID << " for "
                          << current.userName << " (" << current.numberPages << " pages)\n";
            }
        }
        void processNextJob(){
            if(jobQueue.Empty()){
                std::cout << "No jobs to process.\n";
                return;
            }
            Job current = jobQueue.Front();
            std::cout << "Printing Job " << current.jobID << " for "
                      << current.userName << " (" << current.numberPages << " pages)\n";
            jobQueue.Dequeue();
        }
};
```

```cpp

int main(){

    Printer officePrinter(5);
    int choice = 0;
    int nextJobID = 1;

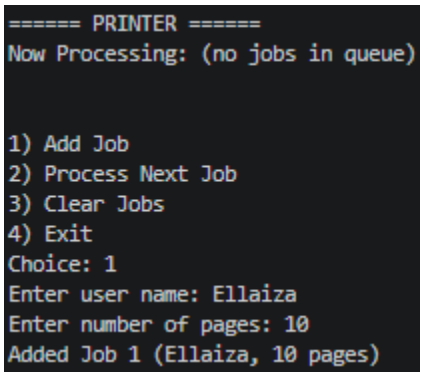
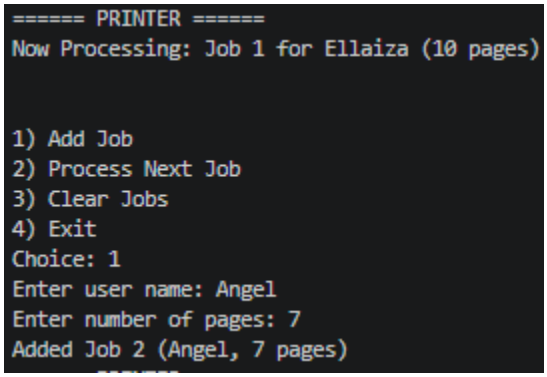
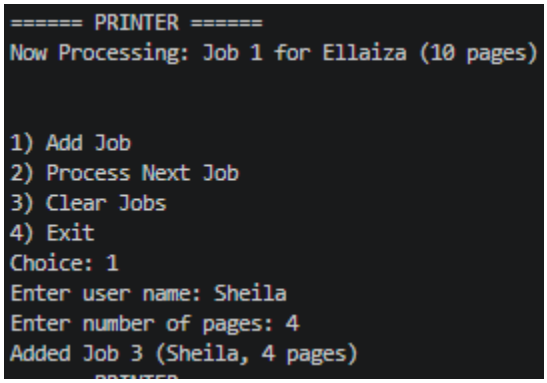
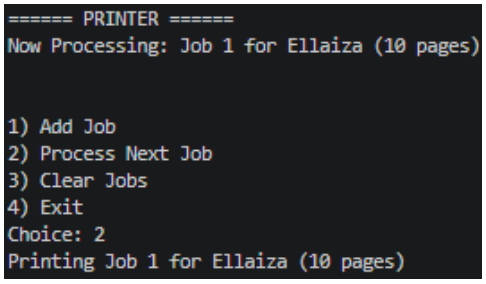
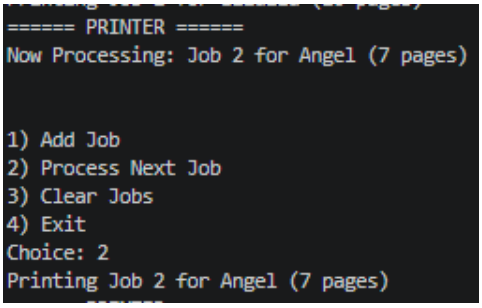
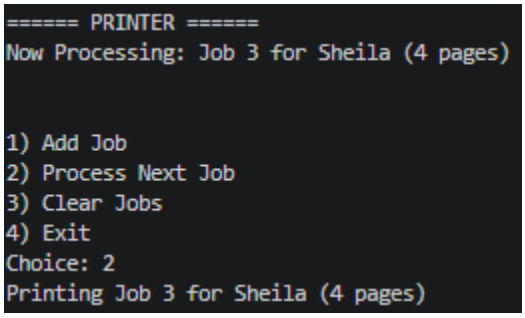
    while(choice != 4){
        std::cout << "====== PRINTER ======\n";
        officePrinter.showJob();
        std::cout << "\n\n"
                  << "1) Add Job\n"
                  << "2) Process Next Job\n"
                  << "3) Clear Jobs\n"
                  << "4) Exit\n"
                  << "Choice: ";
        std::cin >> choice;

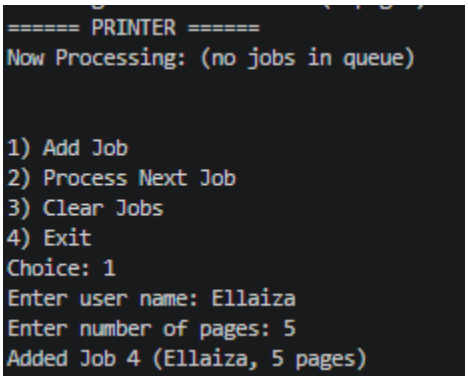
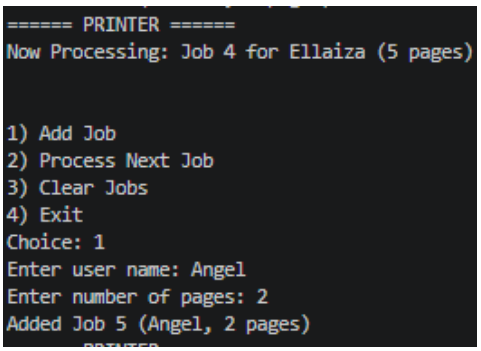
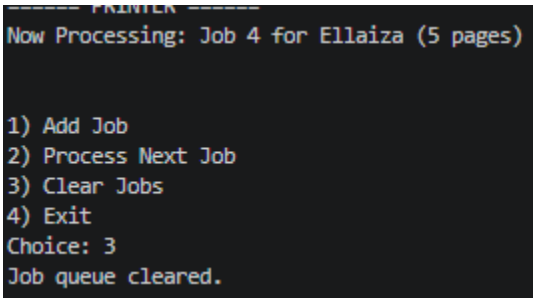
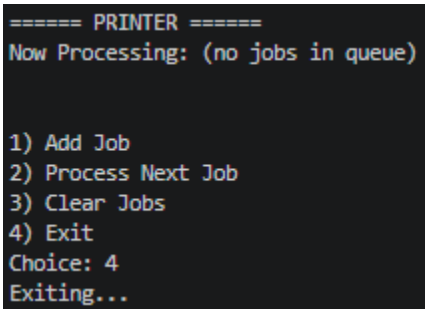
        switch(choice){
            case 1: {
                std::string name;
                int pages;

                std::cout << "Enter user name: ";
                std::cin >> name;
                std::cout << "Enter number of pages: ";
                std::cin >> pages;

                officePrinter.addJob(Job(nextJobID, name, pages));
                nextJobID++;
                break;
            }
            case 2:
                officePrinter.processNextJob();
                break;
            case 3:
                officePrinter.clearJobs();
                std::cout << "Job queue cleared.\n";
                break;
            case 4:
                std::cout << "Exiting...\n";
                break;
            default:
                std::cout << "Invalid choice, try again.\n";
                break;
        }
    }

    return 0;
}
```

Simulation/Output	
Add Job	  
Next Job	  

Clear Job	  
Exit	

Analysis

When I added Ellaiza's job first, then Angel's, then Sheila's, the front of the queue stayed on Ellaiza's job the whole time even after the other two were added, showing that adding a job to the back doesn't touch the front at all, the front only changes when something actually gets removed. When I used "Process Next Job," Ellaiza's job printed first and the front moved to Angel's job, then when I processed again, Angel's job printed and the front moved to Sheila's, which happens because Dequeue() moves q_front forward by one spot each time, so jobs always come out in the same order they went in. This also shows the queue is FCFS, since Ellaiza's job had way more pages (10) than Angel's (6), but it still went first just because it was added first, not because of the page count. I also ran into a bug where Dequeue() was using q_back instead of q_front to figure out the next front position, and after I dequeued Ellaiza's job, the front jumped straight to an empty slot in the array instead of moving to Angel's job, so it printed "Job 0" with no name and 0 pages. This happened because q_back was pointing at Sheila's job (the last one added), so the front got calculated off the wrong index entirely and skipped over Angel's job, and once I fixed it to use q_front instead, the queue went back to processing everyone in the correct order.

Defense: Array or List

I used an array based implementation because it seemed like the easiest one to follow, the logic of tracking a front and back index and moving through the array felt more straightforward to reason about than managing pointers and nodes in a linked list, and within that I used a circular array queue specifically because it seemed more efficient and convenient than a plain array queue. With a regular array queue, once you dequeue from the front, that slot just goes unused, the array can eventually fill up with dead space even if jobs at the back have already been removed, but a circular queue avoids that by wrapping q_front and q_back back to the start of the array using % q_capacity, so freed up slots at the front get reused instead of wasted. This means I do not have to shift every remaining element down after each dequeue, which is more efficient.

Conclusion

In this activity, I learned the difference between using a built in data structure and building one from scratch, since the STL queue was optimized and easy to use but did not provide a way to display all elements, while the self made linked list version showed me what a queue actually does behind the scenes since I had to manage the front and back pointers directly. During the procedure, the STL queue behaved as expected, ELLAIZA came out in the same order it was pushed, and after popping the front and pushing X, the order updated correctly to LLAIZAX, while the linked list queue also worked correctly once I fixed a naming conflict between my front and back pointer variables and my display functions. For the supplemental activity, I built a Printer simulation using the circular array queue, and when I added Ellaiza's, Angel's, and Sheila's jobs, the front stayed on Ellaiza's job until it was processed, then moved to Angel's, then Sheila's, showing the queue is FCFS regardless of page count, though I ran into a bug where Dequeue() used q_back instead of q_front, which caused the front to skip to an empty slot after the first job was removed, and fixing it back to q_front resolved the issue. Overall, I think I did well since I completed all three versions and was able to trace and fix a real bug on my own, but my main area for improvement is being more careful with pointer and index variables, since a simple mix up between q_front and q_back was enough to break the whole queue, so going forward I want to slow down and trace through pointer logic more carefully before running the program.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, summarization, code assistance, explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used AI to adjust my grammar, summarize, helped me with errors, and explained concepts I'm unfamiliar with.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.